

PRESENTACIÓN EJECUTIVA • V1.0

Diseño *conceptual y lógico* de la base de datos

Arquitectura híbrida PostgreSQL + MongoDB para una plataforma e-commerce multivendedor sobre el dataset de Olist (~99k clientes, ~1M geolocalizaciones).

EQUIPO

Sadane Acevedo Virgues
Julian C. Corredor Rojas
Brayan E. Calderón Gómez
Yerlinson Maturana Serna

DOCENTE

Miguel Alfonso Varela

ASIGNATURA

Diseño y Optimización
de Bases de Datos

INSTITUCIÓN • AÑO

Universidad de la Sabana
2026

CONTEXTO

Ecommify es una plataforma e-commerce multivendedor de productos tecnológicos.

Centraliza pedidos, pagos, envíos y reseñas. Debe manejar datos transaccionales de alta consistencia y, a la vez, **datos analíticos de alto volumen y flexibilidad**.

CLIENTES

~99k

+30% anual

PEDIDOS

~99k

+30% anual

ÍTEMES · PAGOS
· RESEÑAS**~320k**

+35–40% anual

PRODUCTOS ·
VENDEDORES**~36k**

+10–20% anual

GEOLOCALIZACIONES

~1.0M

+5% anual · prefijos CP

REQUISITOS

Lo que el sistema debe hacer — y bajo qué garantías.

FUNCIONALES (RF)

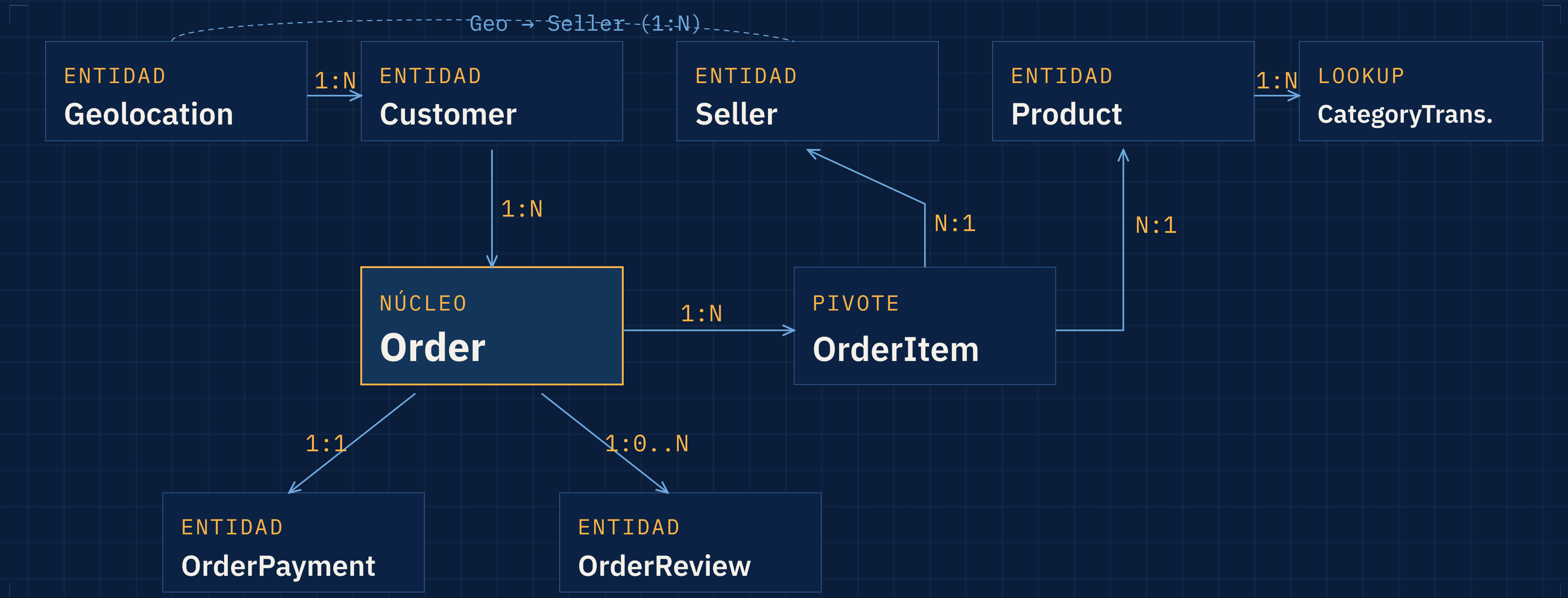
- RF-01** Registro de clientes con información geográfica (ciudad, estado, CP).
- RF-02** Gestión de pedidos y sus estados (pendiente → entregado / cancelado).
- RF-03** Cada pedido contiene múltiples ítems con producto y vendedor.
- RF-04** Pagos por pedido (tipo, cuotas, monto). Pueden ser múltiples y secuenciales.
- RF-05** Reseñas de clientes sobre pedidos (puntaje 1–5, comentario, fecha).
- RF-06** Catálogo con nombre, categoría, dimensiones y fotos.
- RF-07–09** Vendedores geolocalizados, traducción PT→EN de categorías y consulta por CP.

NO FUNCIONALES (RNF)

- RNF-01** Consistencia ACID en pedidos y pagos (PostgreSQL).
- RNF-02** Escalabilidad horizontal del catálogo y reseñas (MongoDB).
- RNF-03** Disponibilidad ≥ 99.5% del tiempo.
- RNF-04** Consultas de pedidos por cliente en < 500 ms.
- RNF-05** Flexibilidad: atributos variables por categoría sin alterar el esquema base.
- RNF-06** Seguridad: control de acceso por roles sobre datos personales.
- RNF-07** Mantenibilidad: normalización mínima 3FN en datos transaccionales.

DISEÑO CONCEPTUAL

Nueve entidades, un núcleo transaccional alrededor de *Order*.



Núcleo transaccional normalizado hasta 3FN, con integridad referencial ACID.

1FN · ATOMICIDAD

Atributos escalares o tipos avanzados nativos (JSONB, arrays, composite, range) tratados como valores atómicos.

2FN · DEPENDENCIAS COMPLETAS

Sin dependencias parciales sobre claves compuestas. `order_items` no duplica atributos del pedido.

3FN · SIN TRANSITIVIDAD

`city` y `state` dependen de `zip_code_prefix` → tabla maestra `geolocation_zip`.

3FN COMO TECHO, NO PISO

No se busca BCNF/4FN — sobre-normalizar degrada OLTP. La desnormalización vive en vistas materializadas.

TABLAS DEL MÓDULO

TABLA	ROL
customers	Compradores · <code>GEOMETRY</code>
sellers	Vendedores con coordenadas
orders	Núcleo · <code>RANGE part.</code>
order_items	PK compuesta · resuelve M:N
payments	Pagos · <code>JSONB gateway</code>
inventory	Stock available / reserved
order_reviews	CHECK score 1-5
product_category_translation	Lookup PT → EN

POSTGRESQL — CAPACIDADES NATIVAS

Optimizar sin romper la integridad relacional.

JSONB `payments.gateway_metadata`

Las pasarelas devuelven estructuras variables. Indexar sin columnas dispersas que estarían nulas la mayoría del tiempo.

TSTZRANGE `orders.delivery_window`

Un único tipo de rango en vez de dos columnas. Valida superposiciones y que inicio < fin a nivel nativo.

PostGIS `customers.location, sellers.location`

`GEOMETRY(Point, 4326)` habilita consultas espaciales como `ST_Distance` para calcular envíos por distancia real.

pgcrypto `gen_random_uuid()`

PK como UUID generadas nativamente. No revela volúmenes transaccionales por IDs secuenciales.

PARTICIONAMIENTO `orders` particionada por `RANGE` sobre `created_at`, en cortes mensuales (`orders_2026_01...`). Separa datos calientes de fríos.

DISEÑO LÓGICO · MÓDULO MONGODB

Esquema dinámico para atributos que varían por categoría.

COLECCIONES PRINCIPALES

products

Catálogo

reviews

Histórico

user_sessions

Analítica

ATTRIBUTE PATTERN

Especificaciones técnicas como arreglo de pares clave–valor (k, v). Un solo índice compuesto, búsquedas eficientes en cualquier atributo, sin esquema rígido lleno de nulls.

SUBSET PATTERN

El documento del producto incrusta solo las 5 reseñas más recientes. Renderizar la ficha del Front-End con una única consulta — reseñas completas viven en **reviews**.

PRODUCTS · DOCUMENTO EJEMPLO

```
// db.products
{
  "_id": "prod_8a2b5c",
  "name": "Yamaha FZ25 Custom Parts",
  "category": {
    "main": "Automotive",
    "sub": "Motorcycle Accessories"
  },
  "technical_specifications": [
    { "k": "material", "v": "Carbon Fiber" },
    { "k": "color", "v": "Matte Black" }
  ],
  "metrics": {
    "average_rating": 4.8,
    "total_reviews": 124
  },
  "recent_reviews": [ /* top 5 */ ]
}
```

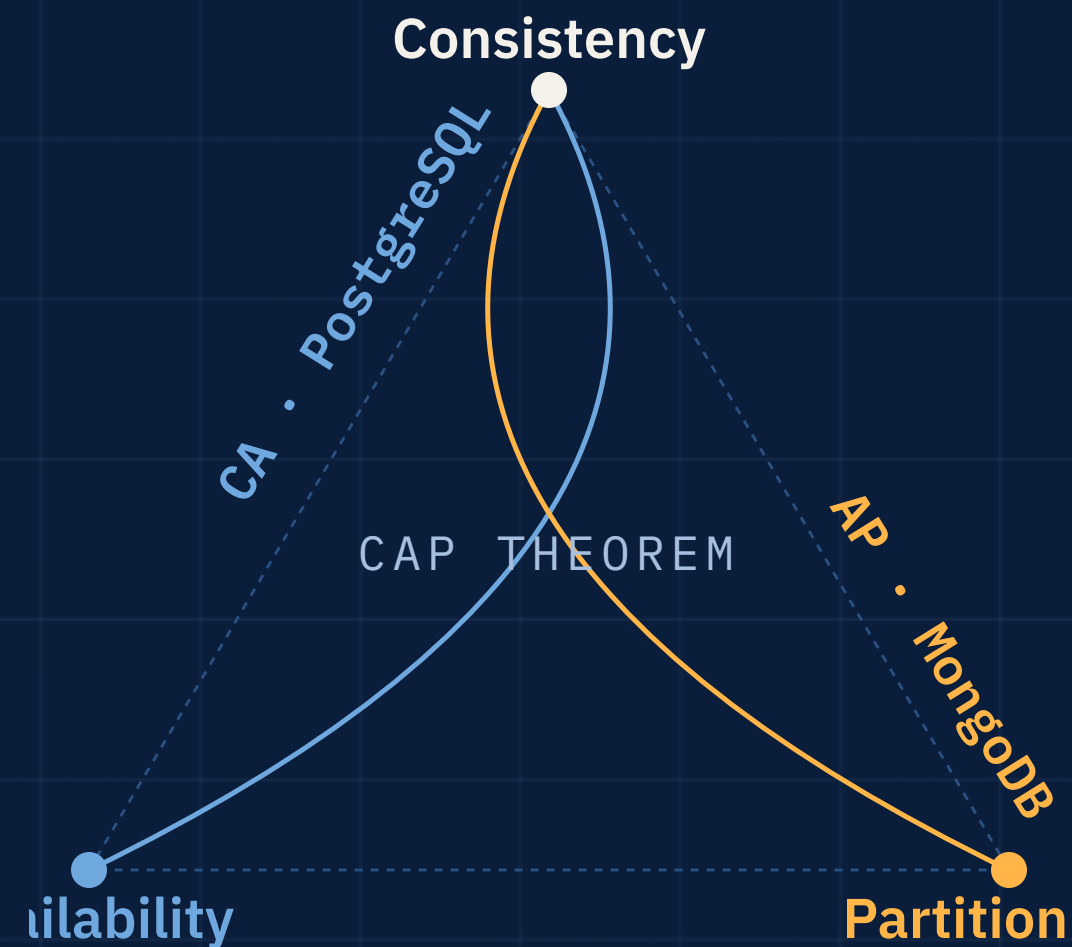
DECISIONES ARQUITECTÓNICAS

Cada entidad va al motor que mejor sirve a su carga de trabajo.

ENTIDAD / MÓDULO	MOTOR	JUSTIFICACIÓN
Clientes	PostgreSQL	Integridad, claves únicas, relaciones con pedidos, direcciones y pagos.
Vendedores	PostgreSQL	Datos transaccionales y relaciones con productos, órdenes e inventario.
Pedidos	PostgreSQL	Consistencia ACID, trazabilidad y control transaccional.
Pagos	PostgreSQL	Alta consistencia, auditoría y validaciones estrictas.
Inventario	PostgreSQL	Evita inconsistencias en stock durante compras concurrentes.
Productos	PG + Mongo	PG guarda datos principales; Mongo almacena atributos variables, imágenes y características.
Reviews	MongoDB	Datos semiestructurados, alto volumen y consultas flexibles.
Catálogo extendido	MongoDB	Atributos dinámicos por categoría sin alterar el esquema relacional.
Analítica / dashboards	PG · vistas materializadas	Reportes periódicos de ventas, clientes y categorías.

TRADE-OFF CENTRAL

CA para el dinero · **AP** para el catálogo.

**PEDIDOS · PAGOS · INVENTARIO**

ACID con atomicidad estricta. Si un pago falla, el inventario no se descuenta. Postura CA: consistencia absoluta de datos financieros sobre tolerancia a particiones.

CATÁLOGO DE PRODUCTOS

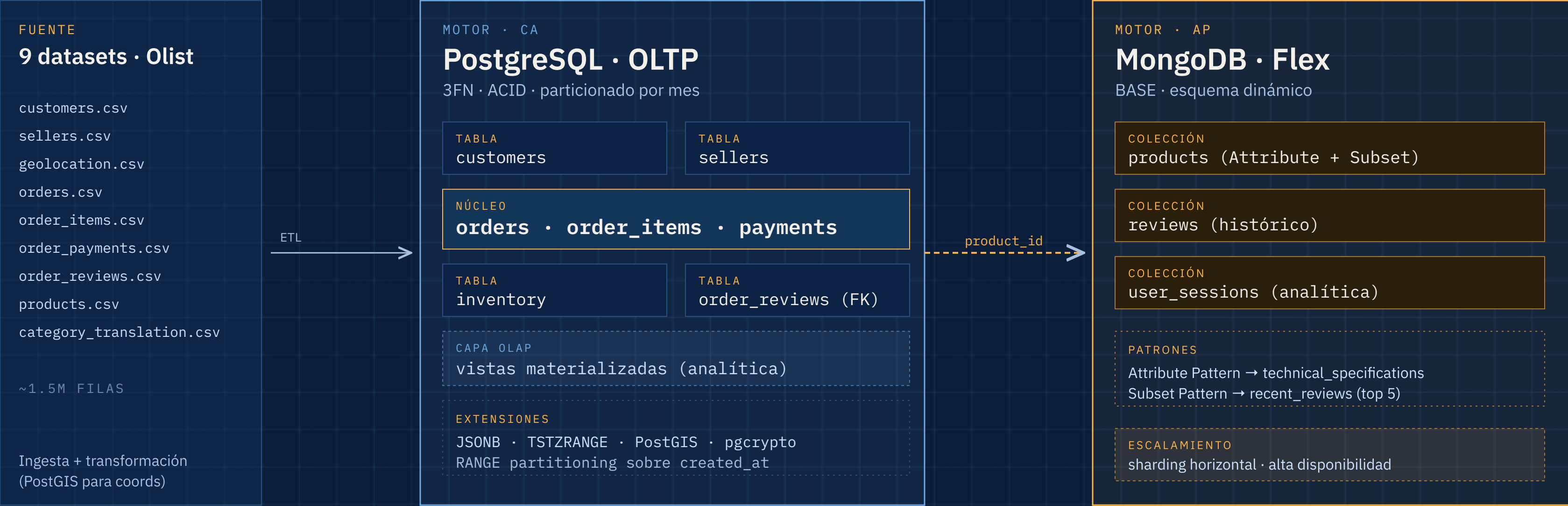
BASE · soft-state · eventual consistency. Postura AP: el catálogo siempre responde, la carga se distribuye horizontalmente.

RESEÑAS & ANALÍTICA

BASE sin riesgo crítico si un dato tarda en propagarse. Postura AP: escalamiento horizontal masivo para picos de tráfico.

TOPOLOGÍA DE DATOS

Dos motores, una sola plataforma — conectados por **product_id**.



TRADE-OFFS

Cada decisión tiene un costo — y una mitigación explícita.

TRADE-OFF	RIESGO	MITIGACIÓN
Usar PostgreSQL + MongoDB complejidad operativa	Duplicidad o desincronización entre motores.	Flujos claros de sincronización y eventos de actualización entre PG y Mongo.
MongoDB con esquemas flexibles cero rigidez	Documentos inconsistentes entre sí.	Validaciones a nivel de aplicación + <code>\$jsonSchema</code> en la colección.
PostgreSQL como verdad analítica cargas OLAP sobre OLTP	Crecen los costos de consultas analíticas.	Particionamiento por mes, índices selectivos y vistas materializadas.
Catálogo dividido SQL / NoSQL sin JOIN directo orders ↔ products	Consultas más complejas, riesgo de incoherencia histórica.	<code>product_id</code> canónico en PG · <code>price congelado</code> en <code>order_items</code> para facturación inmutable.

CIERRE

Cuatro fases para llevar el diseño a producción.



RESULTADO ESPERADO

Una plataforma híbrida donde **el dinero es consistente** y **el catálogo es elástico** — sin comprometer ninguno de los dos.

Gracias.